Elan EAF $\rightleftarrows$ Pangloss XML

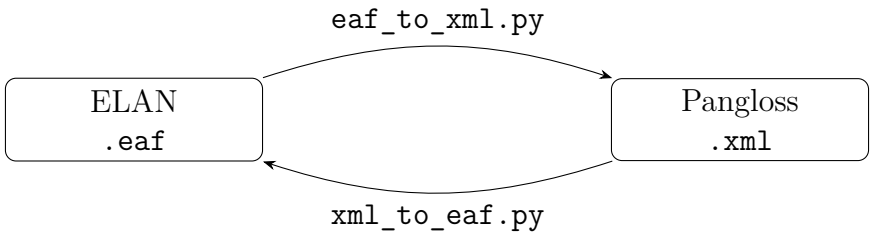
Sara Graça

July 2026

Contents

1 Project Goal

The goal of this project is to create two scripts: one that converts from the Elan `.eaf` format to the Pangloss `.xml` format and one that does the conversion in the opposite direction.



Both scripts are standalone Python 3 files (3.8 or later) with no external dependencies (only the standard library). They are run from the command line:

```
python3 eaf_to_xml.py  <input>  <output>  [options]
python3 xml_to_eaf.py  <input>  <output>  [options]
```

This document has two parts. The first part of each section is a short user manual. The rest is for someone who will work with the scripts themselves.

2 User Manual

Look at file structure. Both scripts have an `--inspect` mode that only prints what is in a file and exits:

```
python3 eaf_to_xml.py input.eaf --inspect
python3 xml_to_eaf.py input.xml --inspect
```

Convert one file. Give an input and an output. The output can be a file name:

```
python3 eaf_to_xml.py input.eaf output.xml
python3 xml_to_eaf.py input.xml output.eaf
```

The output can be a folder (the file keeps its own name and only the extension changes):

```
python3 eaf_to_xml.py input.eaf out/          # -> out/input.xml
python3 xml_to_eaf.py input.xml out/          # -> out/input.eaf
```

The output path is resolved *before* the interview, so an unusable target is reported at once rather than after every question has been answered. An output name not ending in the expected extension is a warning, not an error; the file is still written there.

The script asks a series of questions about which tier plays which role, then converts. At the end it offers to save the answers as a JSON configuration, so the questions never need answering twice for files with the same structure.

Convert a folder. Give a folder as input. Files with the same structure are grouped, and each group is configured once:

```
python3 eaf_to_xml.py corpus_eaf/ corpus_xml/
python3 xml_to_eaf.py corpus_xml/ corpus_eaf/
```

Only the `.eaf` (respectively `.xml`) files directly inside the folder are taken — subfolders are not searched.

Reuse saved answers. `--config` accepts a single JSON file:

```
python3 eaf_to_xml.py ...  ... --config config.json
python3 xml_to_eaf.py ...  ... --config config.json
```

`--config` also accepts a folder of JSON files. With a folder, every input file is matched to the config that fits it, the user confirms the proposed mapping, and everything converts without questions. Files no config fits are configured interactively and their new configs are saved into the same folder:

```
python3 eaf_to_xml.py ...  ... --config configs/
python3 xml_to_eaf.py ...  ... --config configs/
```

Navigation. At any question, type `<` to go back one question. `Enter` skips optional tiers. Typing `y` accepts a suggested tier. For a yes or no question there are the options `[Y/n]` or `[y/N]`; to pick the one in capital letter, press Enter. During a folder interview with various groups of files (grouped by their different structures), typing `<` at the first question of a group returns to the beginning of the questions for the previous group; `Ctrl+C` stops the run and asks whether to convert and save the groups already answered before quitting.

3 Elan EAF → Pangloss XML

`eaf_to_xml.py`

ELAN stores annotations in named *tiers* arranged in a parent–child hierarchy, and imposes no meaning on the names: one corpus can call its transcription `tx`, another `transcription`, another `phono`. Pangloss, by contrast, has fixed roles — sentence, transcription, translation, word, morpheme, gloss. The whole purpose of the interview to the user is to record which tier plays which role.

Command line

`input`	an `.eaf` file, or a folder of them
`output`	an `.xml` file name, or a folder
`--inspect`	print the tier tree and exit (single file only)
`--config PATH`	a saved configuration, or a folder of them

The tier tree

Every run begins by printing the hierarchy:

```
+- 'ref'            type='ref'  lang=''     who='SP'  (259 ann)
   +- 'tx'          type='tx'   lang='khb'  who='SP'  (259 ann)
      +- 'mot'      type='mot'  lang=''     who='SP'  (1204 ann)
         +- 'mb'    type='mb'   lang=''     who='SP'  (2181 ann)
            +- 'ge' type='ge'   lang=''     who='SP'  (2181 ann)
            +- 'ps' type='rx'   lang=''     who='SP'  (2181 ann)
   +- 'ft'          type='ft'   lang='eng'  who='SP'  (258 ann)
```

```
+- 'lit'          type='lit'  lang=''     who='SP'  (0 ann)
```

Each line gives the tier name, its `LINGUISTIC_TYPE`, its language, its speaker and its number of annotations.

The annotation counts are the most reliable clue to a tier's role, and are worth reading before answering anything. A segment tier has roughly one annotation per sentence; a word tier several per sentence; a morpheme tier more still; and a gloss tier normally matches its morpheme tier exactly — `mb` and `ge` both show 2181 above. A count of `0`, as on `lit` above, means the tier exists but was never filled in — selecting it produces empty output.

In this example, the PoS tier is named `ps` but its type is `rx`. Name and type can disagree in real corpora, which is precisely why suggestions read both.

The interview

1. **Output type.** `text` produces `<TEXT>` with sentences; `wordlist` produces `<WORDLIST>` with isolated entries.
2. **Document identifier.** The `id` attribute of the root element; defaults to the file name. Asked in single-file mode only — in folder mode each file takes its own name.
3. **Object language.** An ISO 639-3 code, written as the root's `xml:lang`. This is required: the prompt repeats until a code is given.
4. **Segment tier(s).** The tiers that define the time-aligned units — one per speaker. This is the only tier question that cannot be skipped: without a segment tier there is nothing to build. Candidates are suggested automatically.
5. **Transcription tier(s).** All are selected at once, and each then asked for an optional `kindOf` label (`phono`, `ortho`, …) — pressing `Enter` leaves a form unlabelled. Each becomes a `<FORM>`, and one form at most may go unlabelled, since two plain `<FORM>`s could not be told apart on the way back.
6. **Translation tier(s).** Each becomes a `<TRANSL>` and is asked for its own language code. The language is mandatory once a tier is chosen: `Enter` accepts the suggested language code, and an empty answer is refused.
7. **Notes tier(s).** Become `<NOTE message='...'>`.
8. **Word tier**, and optionally a **word gloss tier**. The first becomes `<W><FORM>` inside each sentence, the second `<W><TRANSL>`. Both questions are skipped entirely for a wordlist, where the entry *is* the word.
9. **Morpheme tier**, and optionally a **morpheme gloss tier** with its language. These become `<M><FORM>` and `<M><TRANSL>`.
10. **Part-of-speech tier**, optional. Pangloss has no element for a part of speech, so it is appended to the morpheme *gloss* with a chosen separator: with `:`, a morpheme glossed `go` whose PoS is `vi` is written `go:vi`. Where a morpheme has no gloss the PoS stands alone, without a leading separator.

The questions are not a fixed list. Each answer can remove later questions: declining the word tier withdraws the word-gloss question, declining the morpheme tier withdraws the

gloss, the gloss language, the PoS tier and its separator.

Tier suggestions

Two stable signals. Tier names vary wildly (`syllabique/mots/gloses`, `A_morph-gls-en`, `phono/mot/morph`). What is stable is the `LINGUISTIC_TYPE` (`ref`, `tx`, `mot`, `mb`, `ge`, `ps/rx`, `ft`) and the hierarchy position: morphemes sit under words, glosses under morphemes. Each suggestion combines a lexical test with a structural one.

The haystack. The lexical test runs against the lower-cased *"name + type"* string, so either can trigger a match — the tier `ps` with type `rx` above is found by the PoS role on its type alone. Short tokens (`ge`, `rx`, `ps`, `mb`, `tx`) are matched as substrings.

Positive and negative keywords. A candidate must match at least one positive keyword and no negative:

Role	Positive	Negative
transcription	tx, txt, transcr, phono, syllab, ortho	word, mot, wrd, morph, mb, gls, gloss, glose, pos, msa, trad, ft, lit, note, not, par, segnum
word	word, mot, wrd	morph, mb, gls, gloss, glose, pos, ps, rx, msa, par, cf, hn, trad, wps
morpheme	morph, mb, mor	gls, gloss, glose, pos, rx, msa, par, cf, hn, type, variant, wps, cat, append, num, segnum
gloss	gls, gloss, glose, ge, meaning	pos, rx, msa, cat, gram, par, cf, hn, type, append, variant, wps
part of speech	pos, msa, gram, cat, tag, rx, ps	gls, gloss, glose, meaning, cf, hn, append, morph-txt, word, mot

The structural constraint. Each role is also constrained by position, evaluated against the tier already chosen for the parent role, so the suggestions cascade:

Role	Must be
transcription	a descendant of the segment tier, *or the segment tier itself*
word	a descendant of the segment tier
word gloss	a direct child of the chosen word tier
morpheme	a descendant of the word tier, or the segment tier if there is none
morpheme gloss	a descendant of the chosen morpheme tier
part of speech	a descendant of the chosen morpheme tier

A transcription may be the segment tier (a flat corpus has no separate reference tier). The word gloss must be a *direct* child so a morpheme gloss two levels down does not qualify.

Ranking. Surviving candidates are ordered by depth (fewest steps up the `PARENT_REF` chain), ties broken by annotation count, most first.

Translations and notes. Translations, notes, and transcriptions are multi-select. Whereas transcription pre-selects only the single best tier, since one is the norm, translations and notes additionally pre-select every qualifying tier. Both need only descend from the segment tier and their keywords are:

Role	Positive	Negative
translation	ft, trad, translation, transl, phrase-gls, phrase-gloss	morph, word-gls, word-gloss, mb, -lit, note, segnum
notes	note, comm, remark, observ	notation, ft, trad, transl, morph, mb, gls, gloss, segnum

Segment tiers. Purely structural, no keywords: a candidate must be time-aligned, parentless, non-empty, and have at least one child tier (the last separates a real segment tier from a standalone track). Grouped by speaker, the winner per speaker is the one with the most child tiers. A dropped speaker is reported:

```
WARNING: detected speaker(s) not in your selection: SP2
Their sentences will be absent from the output.
```

Language codes. Guessed from `LANG_REF`/`DEFAULT_LOCALE` first, else a code appearing as a whole token in the name (`A_phrase-gls-en` → `en`). Token-boundary matching, so `sentence` does not yield `en`.

Finally. Every suggestion is only a default, overridable by number or name, and absent where nothing qualifies.

No silent dropping

The final summary lists the mapping and then, separately, every tier that will *not* appear in the output. When such tiers exist, the script offers to go back:

```
Tiers NOT exported (2): comm, wps
(their content will be absent from the XML)

Adjust the mapping? (y re-opens the last question, from which '<'
steps back one at a time) [y/N]:
```

Answering `y` (or typing `<`) re-opens the last question with every earlier answer intact; `<` then steps back one question at a time.

The same warning is done when a configuration is reused rather than answered. `--config` reports the mismatch in both directions before converting: tiers the configuration names but the file lacks, which will simply be empty, and tiers the file has but the configuration ignores, whose content will not be exported.

Speakers

A multi-speaker file can mark the speaker in one of two ways: a suffix on the tier name (`tx@SP1`, `tx@SP2`), or ELAN's `PARTICIPANT` attribute. `PARTICIPANT` takes precedence when both are present. The suffix is whatever follows the last `@` (for example `tx@SP1-cp` has the speaker code `SP1-cp`).

Once one speaker's tiers have been mapped, the script proposes to mirror that mapping onto the other speakers by substituting the speaker discriminator, so the questions are answered only once. That discriminator is not assumed to be `@SPx`: the transformation is the span that differs between the two segment tier names, once their common prefix and common suffix are set aside. It may be a suffix (`@SP1` vs `@SP2`), a prefix (`A_ref` vs `B_ref`), or anything between — whatever distinguishes one speaker's tiers is applied to every other role, while tiers with no discriminator map to themselves. Where a mirrored tier does not exist, the script says which roles were dropped.

Folder conversion

Given a folder, the script parses every file and groups those belonging to the same corpus template:

```
Found 3 tier structures across 17 files (speaker codes and
optional tiers merged).
```

What counts as the same structure

Two files are grouped when they share a *spine*: the base tiers — speaker suffixes removed — that have at least one child. For the tree above, the spine is `ref`, `tx`, `mot`, `mb`: the backbone the corpus is built on. Leaf tiers such as `ge`, `ps` or a second translation are treated as optional and do not separate two files. (If a corpus hangs anything under `ge`, `ge` joins the spine.) The smaller file's spine must be contained in the other file's spine.

Each tier in the spine is identified by its position, not only its name — `tx` directly under `ref` is a different spine entry from a root-level `tx`. A shared name alone is therefore not enough to group two files. The consequence is that a file with a gloss tier and a file without it can be in the same corpus and share one interview. A one-speaker recording and a two-speaker conversation built on the same template are the same corpus. A wordlist with a wholly different backbone is not, and gets its own group.

Two things to note: files that do not match each other directly can still be grouped through an intermediate file that matches both; a file with no parent–child relations at all has an empty spine, which would technically be contained in everything; such files are therefore only grouped with files having exactly the same tiers.

One merged view

Each group is presented as a single tree, built from the union of all its files, with speaker suffixes stripped. Every tier that exists in *any* file of the group appears, so each role can be chosen once for all of them:

```
+- 'ref'          type='ref'  lang='eng|deu'  who='SP1,SP2,SP'  (818 ann)
   +- 'tx'        type='tx'   lang=''         who='SP1,SP2,SP'  (818 ann)
```

Where the files of a group disagree about a tier's type or language, every value is shown (`lang='eng|deu'`). The `who` column lists every speaker code the tier occurs with, across the group. Where the real tier names cannot be inferred from the node name and the speakers they are listed explicitly on a `tiers:` line beneath.

One case deserves attention. If the same base name occurs at two different places in the hierarchy — a genuine `tx` under `ref`, and an unrelated root-level `tx` — the two do *not* merge, since they are different tiers that happen to share a name. The one with more real tiers behind it keeps the plain name; the other is shown as `tx (root)` or `tx (under …)` and remains selectable separately.

Because the annotation counts in this view are summed across the group, they are useful for comparing tiers with each other but should not be read as a statement about any one file.

From one answer to many files

The interview is answered in base names, and the result is a *template*. At conversion time the template is expanded against each file individually:

- for every speaker code found on that file's tier names, the mapping is stamped out once — `tx` becomes `tx@SP1` for the first speaker, `tx@SP2` for the second, and so on;
- where no suffix exists, the base name is used as-is, and the speaker is read from the segment tier's `PARTICIPANT` if it has one;
- any optional role whose tier that file lacks is simply omitted for that file;
- a file that does not contain the chosen segment tier at all is reported and skipped, rather than silently producing an empty document.

Interrupting and going back

Groups are navigable in the same way as questions. Typing `<` at the first question of a group returns to the beginning of the previous group, which is then answered again from the start.

`Ctrl+C` stops the run:

```
Interrupted after answering 2 structures (11 files).
Convert and save those files before quitting? [Y/n]:
```

Answering yes converts those files and then offers to save their configurations, exactly as a completed run would; answering no writes nothing. The group being answered when the interrupt arrived is not included, since it was never finished.

Reusing a configuration

With `--config` pointing at a single file, that one mapping is applied to every input whose tiers it fits; inputs it does not fit are listed and may be configured by hand. With `--config` pointing at a folder, each input is matched to the configuration whose referenced tiers all exist in that file — the most specific one, where several fit — and the proposed mapping is shown to be confirmed, or adjusted file by file, before anything is written:

```
interview_01.eaf  ->  two_speaker.json
interview_02.eaf  ->  two_speaker.json
wordlist_a.eaf    ->  wordlist.json
```

Matching is by structure — a configuration is never applied to a file it does not fit — but among the configurations that do fit, one named after the file is preferred, so a deliberately file-specific config wins over a general one. One configuration can therefore serve every file built from the same template, with named exceptions where needed. Inputs with no compatible configuration are configured interactively and their configurations are added to the same folder.

The configuration file

Configurations are plain JSON:

`doctype`	`"text"` or `"wordlist"`
`object_lang`	ISO 639-3 code for the root's `xml:lang`; required
`text_id`	document identifier; ignored on reuse, since each file's own name is used
`speakers`	one entry per speaker, each mapping tiers to roles
`who`	code written on each `<S who>`; `""` when the document has a single, unnamed speaker
`segment_tier`	the tier defining the units and their timing
`forms`	list of {`tier`, `kind`}; `kind` becomes `kindOf`, `null` for a plain `<FORM>`
`transl`	translation tiers, in output order
`transl_langs`	language code per translation tier
`notes`	note tiers
`word_form`, `word_gls`	word tier and its gloss
`morph_form`, `morph_gls`	morpheme tier and its gloss
`morph_gls_lang`	language of the morpheme gloss
`morph_pos`	part-of-speech tier
`morph_pos_sep`	separator appended between gloss and PoS

Example:

```
{
  "doctype": "text",
  "object_lang": "khb",
  "text_id": "interview_01",
  "speakers": [
    {
      "who": "SP1",
      "segment_tier": "ref@SP1",
      "forms": [ { "tier": "tx@SP1", "kind": null } ],
      "transl": [ "ft@SP1" ],
      "transl_langs": { "ft@SP1": "eng" },
      "notes": [],
```

```
      "word_form": "mot@SP1",
      "word_gls": null,
      "morph_form": "mb@SP1",
      "morph_gls": "ge@SP1",
      "morph_gls_lang": "eng",
      "morph_pos": "rx@SP1",
      "morph_pos_sep": ":"
    }
  ]
}
```

Configurations written by a folder run are already expanded to each file's real tier names, one per input file, and are directly reusable with `--config`. After a single-file interview, typing `y` saves to `<input name>.json`.

Output format

Rebuilding the annotation graph. ELAN stores annotations as a flat list, so before anything can be written the parent–child structure has to be rebuilt. Children are attached to their parents through `ANNOTATION_REF`, and their order is taken from the `PREVIOUS_ANNOTATION` chain.

Collecting a sentence. The transcription may sit anywhere relative to the segment tier. If it *is* the segment tier its value is used directly; if it lies deeper, all its annotations under that segment are joined with spaces, so even a word-level transcription yields one `<FORM>` per sentence. Units from every speaker are then merged into one sequence ordered by start time.

Times. Read from ELAN's `TIME_ORDER` and written in seconds with three decimals. A converted unit looks like this:

```
<S id="S1" who="SP1">
    <AUDIO start="0.196" end="1.271"/>
    <FORM kindOf="phono">tey dañ nak</FORM>
    <TRANSL xml:lang="eng">today then</TRANSL>
    <W>
        <FORM>dañ</FORM>
        <M>
            <FORM>dañu</FORM>
            <TRANSL xml:lang="eng">go:vi</TRANSL>
        </M>
    </W>
</S>
```

Unit ids. Taken verbatim from the segment tier's own value. A bare number is the exception — it takes the unit tag, `7` becoming `S7`. Where the segment tier has no usable value, or the id it gives is already taken, a running counter supplies `S1`, `S2`, …instead.

Sound file. Taken from the EAF's `MEDIA_DESCRIPTOR`, preferring the relative URL, and only audio is considered. It is written only when exactly one audio file is linked; with several there is no way to tell which is the real recording, so the header is left empty and there is a warning.

Empty units. A unit whose transcription is empty is skipped, and the number skipped is always reported (`93 units (4 empty skipped)`).

4 Pangloss XML → Elan EAF

`xml_to_eaf.py`

This direction is the mirror image of the other. The roles are already known — the XML says what is a sentence, a translation, a morpheme — so nothing needs identifying. What the interview asks instead is what the tiers should be called, and (optionally) which `LINGUISTIC_TYPE` each should be.

Command line

`input`	an `.xml` file, or a folder of them
`output`	an `.eaf` file name, or a folder
`--inspect`	print a summary of the document's content and exit
`--config PATH`	a saved configuration, or a folder of them

Reading the document

Parsing produces a list of units — sentences, or wordlist entries — and a set of content flags: which `kindOf` labels occur on the `<FORM>`s, which languages occur on the `<TRANSL>`s, whether there are notes, words, word glosses, morphemes, morpheme glosses, and which speaker codes appear. Everything afterwards — the proposed names, the grouping of a folder, the per-file adjustments — is derived from those flags. A document is described by what it actually contains.

`--inspect` prints exactly those flags:

```
Document  : TEXT
Text ID   : interview_01
Language  : khb
Sound file: interview_01.wav
Units     : 259  (sentences)

Content:
  Transcription forms    : phono, ortho
  Translations           : 'fra', 'eng'
  Notes                  : yes
  Words                  : yes
  Word glosses           : no
  Morphemes              : yes
  Morpheme glosses       : yes
```

Standard names

The interview opens by proposing a name for exactly the content found:

```
  ref       Utterance id              [XML: <S id> / <W id>]  (time-aligned)
  tx        Transcription (primary)   [XML: <FORM>]
  ft        Free translation (eng)    [XML: <TRANSL>]
  notes     Notes / comments          [XML: <NOTE>]
  mot       Word segmentation         [XML: <W>]
  mb        Morpheme break            [XML: <M><FORM>]
  ge        Morpheme gloss            [XML: <M><TRANSL>]
```

These may be accepted (`standard`) or replaced one by one (`custom`). The scheme has four rules worth knowing:

- each `kindOf` label gets its own transcription tier, named `tx` or named after the label (`phono`, `ortho`, …), if there is more than one;
- a single translation language gives one tier `ft`; several give `ft_fra`, `ft_eng`, …;
- a document with *both* a word gloss and a morpheme gloss cannot call both `ge`; they become `ge_mot` and `ge_mb`;
- words hang under the primary transcription tier and morphemes under the word tier — or directly under the transcription, in a wordlist, where there are no words to speak of.

Recovering a part of speech

Where the morpheme glosses carry a part of speech after a fixed separator — as produced by the other script — the interview offers to split it back out into its own tier (splitting on the last occurence of the separator):

```
Do the morpheme glosses carry a part of speech after a
separator (e.g. 'go:vi')? [y/N]: y
  Separator between gloss and PoS      (press Enter to use ":"):
  Part-of-speech tier name             (press Enter to use "rx"):
```

Linguistic types

By default each tier's type is named after the tier, with two exceptions. All translation tiers share one type, `ft`: they are the same kind of annotation, and the language already lives in the tier name. The two gloss tiers likewise share the type `ge`.

```
  tier ref     ->  type ref   (time-aligned)
  tier ft_fra  ->  type ft    (Symbolic_Association)
  tier ft_eng  ->  type ft    (Symbolic_Association)
  tier ge_mot  ->  type ge    (Symbolic_Association)
  tier ge_mb   ->  type ge    (Symbolic_Association)
```

The constraints are fixed by role and cannot be chosen: the reference tier is time-alignable, transcriptions, translations and glosses use `Symbolic_Association` (one value per parent

annotation), words and morphemes use `Symbolic_Subdivision` (several ordered values per parent).

The summary then shows both columns together:

```
Translation (fra)     : ft_fra  ->  type ft
Morpheme gloss tier   : ge_mb   ->  type ge
```

Speakers

Each unit carries its speaker in the `who` attribute. With two or more speakers, every tier is created once per speaker and suffixed (`ref@SP1`, `tx@SP1`, `ref@SP2`, …). Units with no `who` go on unsuffixed tiers, and a note reports how many.

With exactly one speaker the tiers keep their plain names, and the speaker is recorded in each tier's `PARTICIPANT` attribute instead.

Folder conversion

Files are grouped by *content shape*, and each group is configured once. Two files share a group when they have the same document type and their sets of `kindOf` labels and translation languages are comparable — one side may have extras. Which speakers occur, and whether notes, words, morphemes or glosses happen to be present, never separates two files.

The group's representative is the concatenation of its files' units, so the interview sees the union of everything its files contain. Two situations then make the proposed names true for some files and not others, and the interview says so explicitly rather than letting the display mislead:

Speakers across a group

Where the files disagree about their speakers, the note reads:

```
NOTE: 3 speakers occur across this group (SP1, SP2, SP); no single
      file need have them all.  Each tier below is created once per
      speaker, for the speakers that file actually has.
```

Glosses across a group

The union may force the two-name scheme even though some files carry only one kind of gloss. Those files use a single name instead — and a word-only file and a morpheme-only file are different situations, which may deserve different names. Each is therefore asked separately, and only when that case actually occurs in the group:

```
Gloss tier name in files with only a WORD gloss      [ge]:
Gloss tier name in files with only a MORPHEME gloss  [ge]:
```

The names shown for the group are then annotated with what will really happen:

```
ge_mot    Word gloss       [XML: <W><TRANSL>]  <- 'ge' in files with
                                                  only a word gloss
```

Occasionally no single file has both glosses, yet the group as a whole has each: one file glosses words, another glosses morphemes. The union still forces two names, and the note says as much rather than referring to files that do not exist.

Groups may be revisited after being answered, and `Ctrl+C` behaves as in the other script: the groups already answered can be converted and their configurations saved before quitting.

The configuration file

`ref_tier`	the time-aligned tier holding the unit ids
`forms`	one entry per `kindOf` label: {`kind`, `tier`}
`transl_tiers`	tier name per translation language
`types`	`LINGUISTIC_TYPE` per tier; a tier absent from this map takes its own name as its type
`notes_tier`	tier for `<NOTE>` content
`word_tier`, `word_gls_tier`	word tier and its gloss
`morph_tier`, `morph_gls_tier`	morpheme tier and its gloss
`word_gls_only_tier`	gloss name used by a file that has only a word gloss, in a group where others have both
`morph_gls_only_tier`	the same, for morpheme-gloss-only files
`morph_pos_tier`	tier created by splitting the PoS off the morpheme gloss, or `null` for no split
`morph_pos_sep`	the separator to split on

Example:

```
{
    "ref_tier": "ref",
    "forms": [ { "kind": "phono", "tier": "tx" },
             { "kind": "ortho", "tier": "ortho" } ],
    "transl_tiers": { "fra": "ft_fra", "eng": "ft_eng" },
    "notes_tier": "notes",
    "word_tier": "mot",
    "word_gls_tier": null,
    "morph_tier": "mb",
    "morph_gls_tier": "ge",
    "word_gls_only_tier": "ge",
    "morph_gls_only_tier": "ge",
    "morph_pos_tier": "rx",
    "morph_pos_sep": ":",
    "types": { "ref": "ref", "tx": "tx", "ortho": "ortho",
             "ft_fra": "ft", "ft_eng": "ft",
             "mot": "mot", "mb": "mb", "ge": "ge" },
}
```

Output format

Times. Read from `<AUDIO start end>` in seconds and converted to milliseconds. Each boundary gets its own `TIME_SLOT` even where two annotations meet at the same millisecond: sharing a slot would mean that dragging one boundary in ELAN silently moved its neighbour too. A document with no timing at all still produces a usable file — consecutive one-second placeholder spans are synthesised, with a warning, so ELAN opens it and the annotations can be aligned by hand.

The tier skeleton. The reference tier is the only time-aligned one, and carries the unit ids as its values. Every other tier refers to its parent, chained with `PREVIOUS_ANNOTATION` where the constraint is a subdivision, so that words and morphemes keep their order.

Collapsing the reference tier. Giving the reference tier and the primary transcription the same name merges them: the time-aligned tier then carries the transcription itself instead of the unit ids.

Language. The object language is written as `LANG_REF` on the primary transcription tier. Each translation tier carries its own `LANG_REF`, and every code used is declared in a `<LANGUAGE>` element.

5 The two formats side by side

Role	Pangloss XML	ELAN
document	`<TEXT>` / `<WORDLIST>`, with `id` and `xml:lang`	the file itself
sound file	`<HEADER><SOUNDFILE href>`	`MEDIA_DESCRIPTOR`
unit	`<S id who>` / `<W id>`	annotation on the segment tier
timing	`<AUDIO start end>`, seconds	`TIME_SLOT`s, milliseconds
transcription	`<FORM kindOf>`	one tier per kind
translation	`<TRANSL xml:lang>`	one tier per language
note	`<NOTE message>`	notes tier
word	`<W>` inside `<S>`	word tier, subdivision
word gloss	`<W><TRANSL>`	word-gloss tier
morpheme	`<M><FORM>`	morpheme tier, subdivision
morpheme gloss	`<M><TRANSL>`	morpheme-gloss tier
part of speech	inside the morpheme gloss	a tier of its own
speaker	`who` on the unit	`@code` suffix, or `PARTICIPANT`

The asymmetries in this table are the source of every subtlety in the two scripts. ELAN names tiers freely and Pangloss does not, which is why one direction needs an interview about roles and the other about names. Pangloss has no element for a part of speech, which is why it must travel inside the morpheme gloss. And ELAN can express a speaker in two ways, which is why both must be read.